

구슬 기계

마디나는 IOI 참가자들이 재미있어 할 구슬 기계를 만들었다. 기계의 내부 구조는 아래와 같다:

- 기계는 0번부터 $N - 1$ 번까지 번호가 붙은 N 개의 **노드**로 구성된다. 노드 $N - 1$ 은 **루트**라고 부른다.
- 각 노드 u ($0 \leq u < N - 1$)에 대해서, 노드 u 의 **부모**는 노드 $P[u]$ ($P[u] > u$)이고, 노드 u 는 노드 $P[u]$ 의 자식이다. 루트는 부모가 없다. 자식이 없는 노드는 **리프**라고 부른다.

당신은 N 의 값과 모든 노드 u 의 부모 $P[u]$ 를 **모른다**. 대신, 마디나는 리프의 개수 M 을 알려준다. 또, 리프들의 번호가 $0, 1, \dots, M - 1$ 임도 알려준다. 당신이 해야 할 일은 기계의 동작을 통해서 그 내부 구조를 알아내는 것이다.

기계의 각 노드는 최대 1개의 **구슬**을 저장할 수 있다. 각 구슬은 음수가 아닌 정수 **가치**를 가진다. 어떤 노드에 가치 x 인 구슬이 저장된 경우 그 노드의 가치가 x 라고 말한다. 어떤 노드에 구슬이 저장된 경우 그 노드는 **찻다**고 하고, 그렇지 않은 경우 **비었다**고 한다. 초기에 모든 노드는 비었다.

당신이 할 수 있는 작업은 아래 두가지이다.

- **insert(U, X)**: 가치 X 인 구슬을 저장하는 것을 리프 U 에서부터 시도한다.
 - 리프 U 가 찻 경우, 작업은 `false`를 리턴하고 구슬은 저장되지 않는다.
 - 리프 U 가 빈 경우 구슬은 우선 U 에 저장된다. 그 다음, 구슬은 부모가 비어 있다면 그 부모로 이동한다. 그러한 이동이 불가능할 때까지 이는 반복된다. 즉, 구슬의 움직임은 구슬이 루트에 도달하거나 찻 부모를 만날 때까지 계속된다. 이 작업은 `true`를 리턴한다.
- **collect()**: 루트가 빈 경우 (즉, 기계 전체가 빈 경우), `collect`는 빈 배열을 리턴한다. 그렇지 않은 경우, 아래에 설명된 `traverse` 함수가 호출되어 기계에 존재하는 모든 구슬들의 가치가 배열 S 에 저장된다. `traverse` 함수 호출이 시작될 때 S 는 빈 배열이고 첫 호출은 `traverse(N - 1)`이다.

`traverse(u)`:

노드 u 에 저장된 구슬의 가치를 S 의 끝에 추가한다

$c[u]$ 가 u 의 자식들 중 찻 노드들의 리스트라고 하자

$c[u]$ 를 저장된 구슬의 가치가 단조증가(비감소)하는 순서로 정렬한다

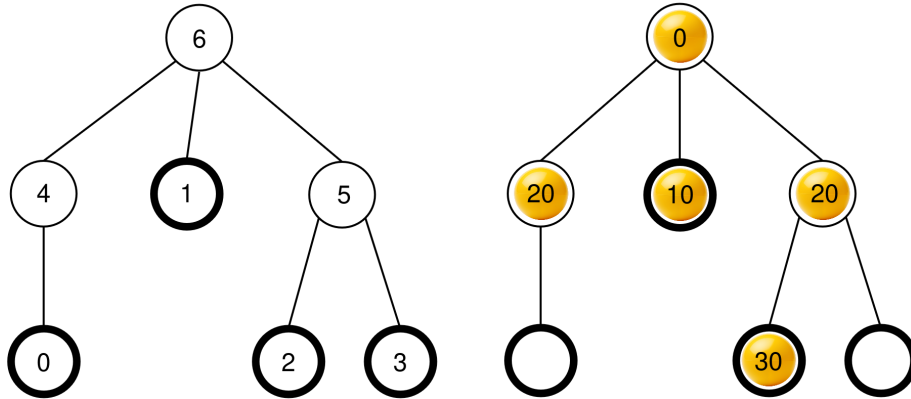
(두개 이상의 구슬이 동일한 가치를 가진 경우 임의의 순서로 등장할 수 있다)

$c[u]$ 의 각 v 에 대해:

`traverse(v)`

이 함수가 끝난 후 기계의 모든 구슬들이 제거되고 collect는 S 를 리턴한다.

예를 들어, $N = 7$ 이고 $P = [4, 6, 5, 5, 6, 6]$ 인 경우를 생각하자. 아래 그림의 왼쪽은 각 노드의 번호가 표시된 빈 기계이다. 아래 그림의 오른쪽은 동일한 기계에 몇 번의 insert 작업이 일어난 이후의 상태를 보여준다. (실제 작업들은 Example 섹션에서 볼 수 있다.)



`collect()`가 호출되었을 때, 루트(노드 6)은 찾다. 따라서, S 는 $S = []$ 로 초기화되고 `traverse(6)`이 호출된다.

traverse(6): 노드 6의 가치는 0이다; S 의 뒤에 그 가치를 추가하면 $S = [0]$ 이 된다. 노드 6의 자식들은 4, 1, 5이고 모두 찾다. 자식 노드들의 가치는 각각 20, 10, 20이다. 단조증가 순서로 정렬하면 $c[6] = [1, 5, 4]$ 가 된다. (노드 4와 5의 가치가 같으므로 $c[6] = [1, 4, 5]$ 도 가능함에 주의하라.) 이제 함수는 $c[6]$ 의 각 노드에 순서대로 `traverse`를 호출한다.

- **traverse(1):** 노드 1의 가치는 10이다; S 의 뒤에 추가하면 $S = [0, 10]$ 이 된다. 노드 1은 찬 자식이 없으므로 추가 호출은 없다.
- **traverse(5):** 노드 5의 가치는 20이다; S 의 뒤에 추가하면 $S = [0, 10, 20]$ 이 된다. 노드 5의 자식 중 노드 2만이 찾다. 따라서, $c[5] = [2]$ 가 되고 `traverse(2)`가 호출된다.
 - **traverse(2):** 노드 2의 가치는 30이다; S 의 뒤에 추가하면 $S = [0, 10, 20, 30]$ 이 된다. 노드 2는 찬 자식이 없으므로 추가 호출은 없다.
 - 이제 `traverse(5)`로 리턴되고, 이 호출은 $c[5]$ 의 모든 찬 자식에 대한 호출을 마쳤으므로 종료된다.
- **traverse(4):** 노드 4의 가치는 20이다; S 의 뒤에 추가하면 $S = [0, 10, 20, 30, 20]$ 이 된다. 노드 4는 찬 자식이 없으므로 호출은 종료된다.

모든 호출이 종료되었고 더 이상 처리할 노드가 없다. 최종적으로 기계의 모든 구슬이 제거되고 collect는 $S = [0, 10, 20, 30, 20]$ 을 리턴한다.

당신이 할 일은, 노드 개수 N 을 파악하고 기계의 구조를 표현하는 부모들의 배열 $R = [R[0], R[1], \dots, R[N-2]]$ 을 리턴하는 것이다. 단, 루트와 리프를 제외한 노드들의 번호는 바꿀 수 있다. 자세히 설명하면, 아래의 조건을 만족하도록 각 노드에 서로 다른 레이블 $L[u]$ ($0 \leq L[u] < N$)를 부여하는 것이 가능한 경우 R 은 정확한 것으로 간주된다.

- 모든 $u(0 \leq u < M, u = N - 1)$ 에 대해 $L[u] = u$ 이고
- 모든 $u(0 \leq u < N - 1)$ 에 대해 $L[P[u]] = R[L[u]]$.

$R[u] > u$ 인 조건을 만족할 필요는 없다. 최종 답을 제출할 때 기계는 비어 있어야 한다.

collect 작업의 횟수를 K , 모든 insert에서 구슬 가치의 최대값을 B , $C = K + B$ 라고 하자. C 는 1000을 넘을 수 없다. 일부 서브태스크는 C 의 값에 따라 다른 점수를 받을 수 있다.

Implementation Details

다음 함수를 구현해야 한다.

```
std::vector<int> find_structure(int M)
```

- M : 기계의 리프 갯수.
- 각 테스트케이스에 대해 정확히 한번 호출된다.
- 함수는 길이 $N - 1$ 인 배열 $R = [R[0], R[1], \dots, R[N - 2]]$ 을 정확한 값들로 채워 리턴해야 한다.

기계를 다루기 위해 아래 두 함수를 호출할 수 있다.

첫번째 함수는:

```
bool insert(int U, int X)
```

- U : 구슬 저장을 시작할 노드의 번호. $0 \leq U < M$ 이 성립해야 한다.
- X : 구슬의 가치. $0 \leq X \leq 1000$ 이 성립해야 한다.
- 구슬이 성공적으로 저장된 경우 이 함수는 true를 리턴한다. 리프 U 가 이미 찬 경우 false가 리턴된다.
- 이 함수는 최대 500 000번 호출할 수 있다.

두번째 함수는:

```
std::vector<int> collect()
```

- 모든 저장된 구슬을 모으고 기계를 비운 후 수집된 배열 S 를 리턴한다.

당신의 솔루션의 실행 과정에서 C 가 1000을 초과하면, 그 시점에 제출 결과로 Output isn't correct: Too many resources used를 받는다.

기계의 구조는 find_structure이 호출되기 전에 확정되어 있다. 그레이더는 결정론적이다. 즉, 동일한 입력에 대해 동일한 작업을 수행한 경우 collect의 리턴은 동일하다.

Constraints

- $2 \leq N \leq 1000$

- $1 \leq M \leq 200$
- $M < N$

Subtasks

Subtask	Score	Additional Constraints
1	5	루트가 정확히 M 개의 자식을 가진다.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	추가적인 제한이 없다.

서브태스크 3과 4에서, 당신이 받는 점수는 C 의 값에 따라 아래와 같이 계산된다.

Subtask 3 (25점)

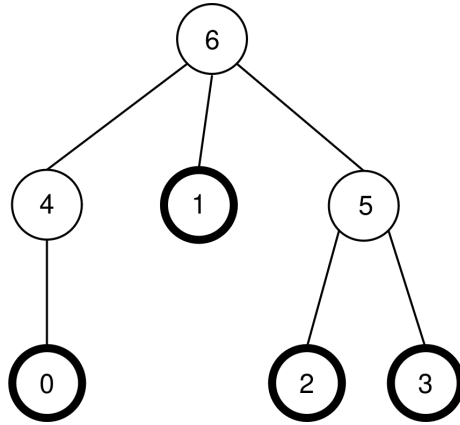
Limits	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask 4 (60점)

Limits	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Example

위 지문과 동일한 기계를 고려하자. 즉, $N = 7, M = 4, P = [4, 6, 5, 5, 6, 6]$ 이다. 아래 그림은 기계를 다시 보인 것이다.



그레이더는 다음 호출을 한다:

```
find_structure(4)
```

이 함수는 아래의 호출들을 할 수 있다.

1. **insert(0, 0)**: 리프 0이 비었으므로, 가치 0인 구슬이 리프 0에 저장된다. 부모 $P[0] = 4$ 이 비었으므로 구슬은 노드 4로 이동한다. 부모 $P[4] = 6$ 이 비었으므로 구슬은 노드 6으로 이동한다. 노드 6은 루트이므로 구슬은 더 이상 이동하지 않는다. 호출은 true를 리턴한다.
2. **insert(3, 20)**: 리프 3이 비었으므로, 가치 20인 구슬이 리프 3에 저장된다. 부모 $P[3] = 5$ 이 비었으므로 구슬은 노드 5로 이동한다. 부모 $P[5] = 6$ 이 찼으므로 구슬은 더 이상 이동하지 않는다. 호출은 true를 리턴한다.
3. **insert(1, 10)**: 리프 1이 비었으므로 가치가 10인 구슬이 리프 1에 저장된다. 부모 $P[1] = 6$ 이 찼으므로 구슬은 노드 1에서 움직이지 않는다. 호출은 true를 리턴한다.
4. **insert(2, 30)**: 리프 2가 비었으므로 가치 30인 구슬이 리프 2에 저장된다. 부모 $P[2] = 5$ 가 찼으므로 구슬은 노드 2에서 움직이지 않는다. 호출은 true를 리턴한다.
5. **insert(1, 25)**: 리프 1이 찼으므로 구슬은 노드 1에 저장되지 않는다. 호출은 false를 리턴한다.
6. **insert(0, 20)**: 리프 0이 비었으므로 가치 20인 구슬이 리프 0에 저장된다. 부모 $P[0] = 4$ 가 비었으므로 구슬은 노드 4로 이동한다. 부모 $P[4] = 6$ 이 찼으므로 구슬은 더 이상 이동하지 않는다. 호출은 true를 리턴한다.

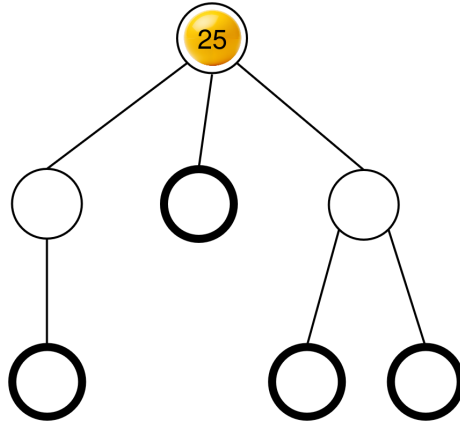
기계의 최종 상태는 위 지문의 그림과 같다.

다음 호출을 한다:

```
collect()
```

지문에서 설명한 바와 같은 과정으로 호출은 $S = [0, 10, 20, 30, 20]$ 을 리턴한다. 직후, 기계는 빈 상태가 된다.

다음, **insert(2, 25)**을 호출한다. 리프 2는 비었으므로 가치 25인 구슬이 리프 2에 저장된다. 구슬은 노드 5로, 또 노드 6으로 이동하고 멈춘다. 이 호출은 true를 리턴한다. 이 때의 기계 상태를 아래 그림에 보았다.



마지막으로 `collect()`를 호출하면 $S = [25]$ 가 리턴되고 기계는 빈 상태가 된다.

`find_structure`가 리턴할 수 있는 정확한 부모 배열은 두가지가 있다: $R = P = [4, 6, 5, 5, 6, 6]$ (레이블을 $L = [0, 1, 2, 3, 4, 5, 6]$ 로 한 것에 해당)과 $R = [5, 6, 4, 4, 6, 6]$ (레이블을 $L = [0, 1, 2, 3, 5, 4, 6]$ 로 한 것에 해당). 이 경우에 $K = 2$ 이고 $B = 30$ 이므로 $C = 32$ 이다.

Sample Grader

Input format:

```

N M
P[0] P[1] P[2] ... P[N-2]

```

Output format:

```

K B C
Q
R[0] R[1] R[2] ... R[Q-1]

```

여기서 Q 는 `find_structure`가 리턴한 R 의 크기이다.